

Scaling from 0 to 1 Million Users — Complete System Design

Load Balancer · DB Replication · Caching · CDN
Message Queues · Sharding · Multi-Region · Fault Tolerance

TOPICS COVERED

1. Single Server → Decoupled Servers

2. Load Balancers (NGINX, HAProxy, ALB)

3. DB Replication (Master-Slave)

4. Caching (Redis, Memcached, Strategies)

5. Message Queues (Kafka, RabbitMQ, SQS)

6. CDN (Cloudflare, CloudFront, Akamai)

7. DB Sharding (Range, Hash, Tools)

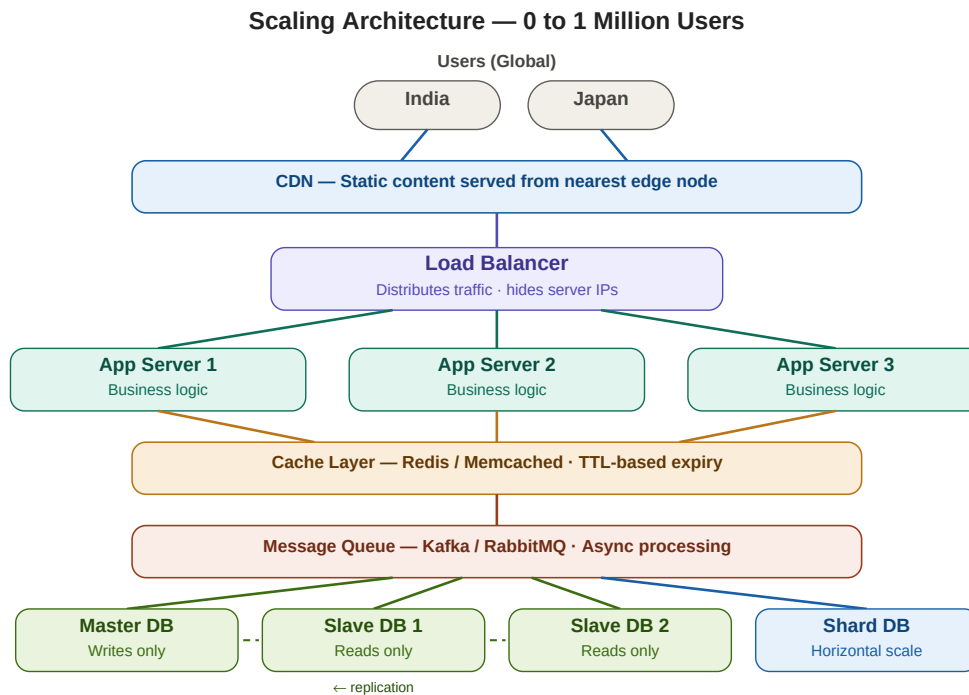
8. Multi-Region & Fault Tolerance

Table of Contents

1. Scaling Journey Overview	3
2. Step 1–2: Single Server → Decoupled Architecture	3
3. Step 3: Load Balancer	4
4. Step 4: Database Replication	5
5. Step 5: Caching Layer	6
6. Step 6: Content Delivery Network (CDN)	7
7. Step 7: Multi-Region & Fault Tolerance	8
8. Step 8: Message Queues	8
9. Step 9: Database Scaling — Sharding	10
10. Quick Revision & Interview Questions	11

1. Scaling Journey Overview

Scaling a system from zero to one million users requires an **incremental architectural evolution**. No system starts fully distributed — you add complexity only when the previous layer becomes a bottleneck. The 9-step journey below shows how each layer is added.



Full scaling architecture — each layer added incrementally as user load grows

Step	What Changes	Why
1	Single server — app + DB together	Zero users, simplest possible setup
2	Separate app server and DB server	Independent scaling and maintenance
3	Add Load Balancer	Single server can't handle traffic spikes
4	Database replication	Read load too high; one DB is a bottleneck
5	Add Cache layer	DB queries are expensive; reduce latency
6	Add CDN	Global users face high latency for static assets
7	Multi-region datacenters	One region goes down; global latency still high
8	Message Queue	Synchronous processing can't handle burst load
9	DB Sharding	Single DB (even replicated) hits write limits

2. Step 1–2: Single Server → Decoupled Architecture

Step 1 — Single Server

Everything on one machine: web server, application logic, and database. Works for a college project or new startup with **fewer than ~1,000 users**. Any bottleneck in one component affects everything.

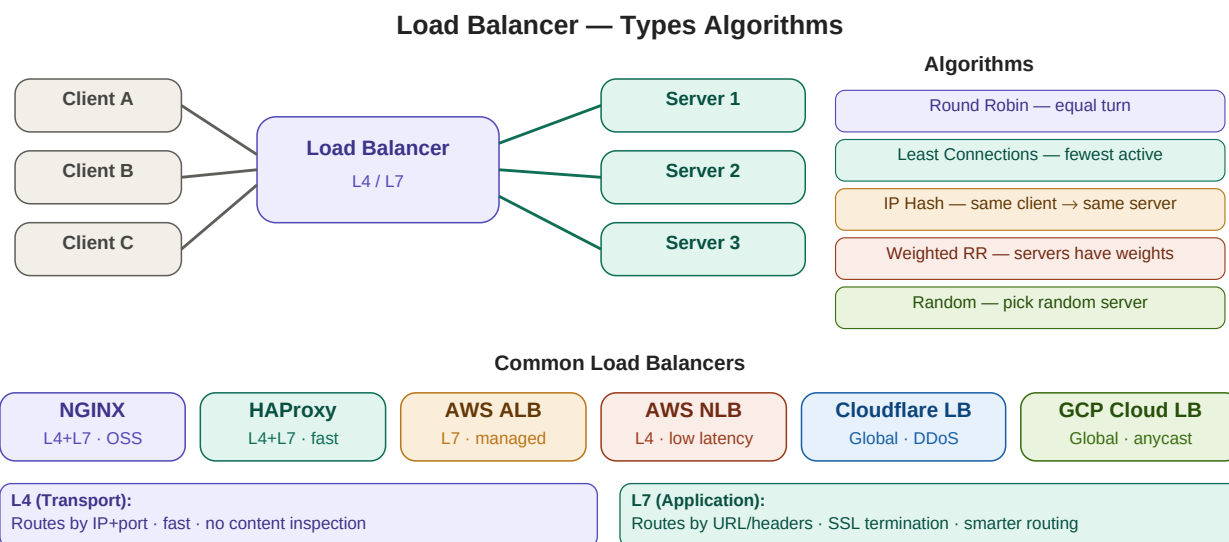
Step 2 — Decouple App Server and DB Server

Split the monolith into two layers. The **application server** handles business logic (HTTP requests, auth, processing). The **database server** handles persistent storage. Each can now be scaled, monitored, and upgraded independently.

- App server CPU-intensive tasks scale separately from DB I/O-intensive tasks
- DB server can use high-memory, high-disk hardware; app server can use many small instances
- Security: DB server not directly exposed to internet
- Allows different teams to own each layer independently

3. Step 3: Load Balancer

When traffic exceeds what one app server can handle, a **load balancer** sits in front of a pool of app servers and distributes requests. It hides backend IPs from the public and reroutes traffic away from failed servers.



Load balancer distributes requests using algorithms · L4 vs L7 determines routing intelligence

Load Balancing Algorithms

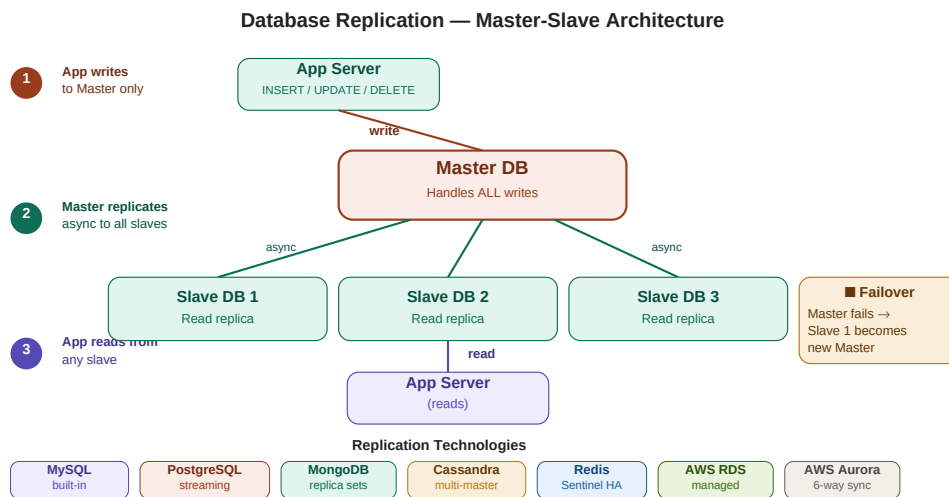
Algorithm	How it works	Best for
Round Robin	Requests cycle through servers equally	Stateless apps, equal-capacity servers
Least Connections	Next request to server with fewest active connections	Long-lived connections, varied load
IP Hash	Same client IP always → same server	Session-sticky apps, WebSockets
Weighted Round Robin	Servers get proportional share by weight	Mixed hardware capacities
Random	Pick a random server	Simple, high-volume stateless requests

Load Balancer Technologies

Tool	Type	Pros	Cons	Best for
NGINX	Software L4+L7	Free, fast, reverse proxy, TLS	Manual config	General purpose, APIs
HAProxy	Software L4+L7	Extremely fast, battle-tested	Complex config	High-throughput TCP/HTTP
AWS ALB	Managed L7	Zero ops, path/host routing, WAF	AWS only, cost	AWS apps, microservices
AWS NLB	Managed L4	Ultra-low latency, static IP	No HTTP features	TCP/UDP, gaming, IoT
Cloudflare LB	Global L7	DDoS protection, global anycast	Paid	Public-facing APIs, DDoS risk
GCP Cloud LB	Global L7	Global anycast, auto-scaling	GCP only	Google Cloud workloads
Traefik	Software L7	Docker/K8s native, auto-discovery	Complex for simple cases	Containers, microservices

4. Step 4: Database Replication

A single database becomes a read bottleneck at scale. **Master-Slave replication** routes all writes to one master and distributes reads across multiple slave replicas. Slaves asynchronously sync from master.



Master handles all writes · Slaves serve reads · Async replication · Auto-failover on master failure

Replication Technologies — Deep Comparison

Technology	Replication Type	Consistency	Failover	Best for
MySQL + InnoDB	Async / Semi-sync	Eventual (async)	MHA / Orchestrator	OLTP, web apps
PostgreSQL	Streaming (WAL-based)	Near-real-time	Patroni / pg_auto_failover	Complex queries, JSONB
MongoDB	Replica Sets (3+ nodes)	Eventual / tunable	Automatic (Raft-based)	Document data, flexible schema
Cassandra	Multi-master (leaderless)	Tunable (quorum)	Built-in (no SPOF)	Write-heavy, global scale
AWS RDS	Managed async/sync	Sync (Multi-AZ)	Automatic (<60s)	Managed MySQL/Postgres
AWS Aurora	Shared storage cluster	6-way sync replication	< 30s automatic	High availability OLTP
Redis Sentinel	Async master→slave	Eventual	Sentinel monitors + promotes	Cache HA, session store

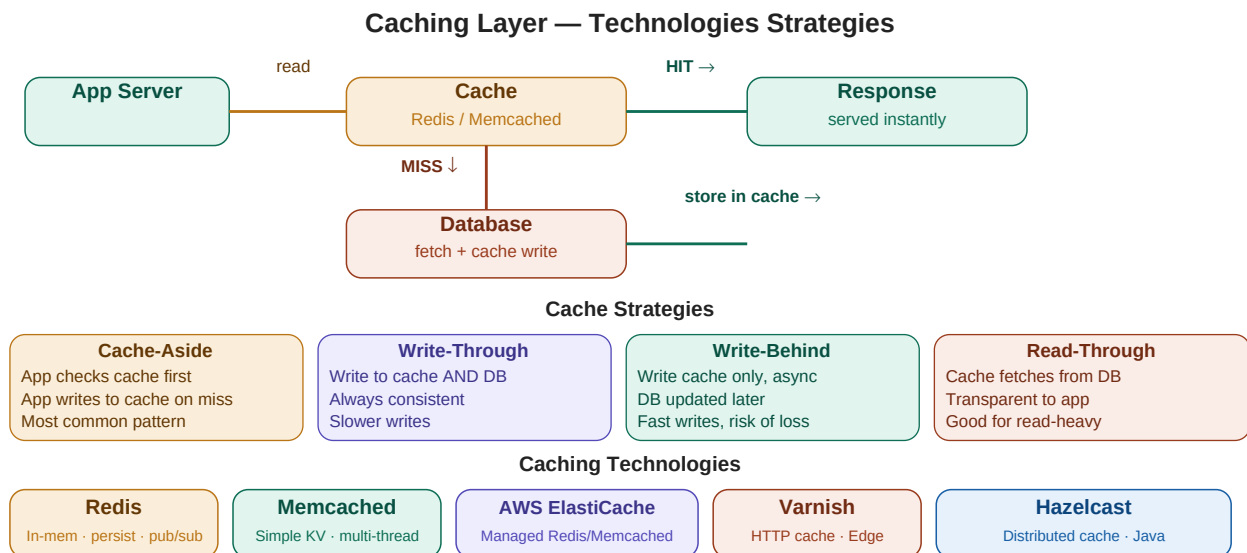
Read/Write split: route all INSERT/UPDATE/DELETE to master, all SELECT to slaves.

Replication lag: slaves may be milliseconds behind master — avoid reading your own write from slave.

Failover: if master dies, a slave is promoted. Auto-failover tools: Orchestrator (MySQL), Patroni (Postgres), MongoDB Replica Sets.

5. Step 5: Caching Layer

Database queries are expensive — disk I/O, network, query parsing. A **cache** stores frequently accessed data in memory, returning results in microseconds instead of milliseconds. The app checks cache first; on a miss it queries DB and populates the cache.



Cache-hit path skips DB entirely · Cache miss fetches from DB and stores result · TTL expires stale data

Caching Technologies — Deep Comparison

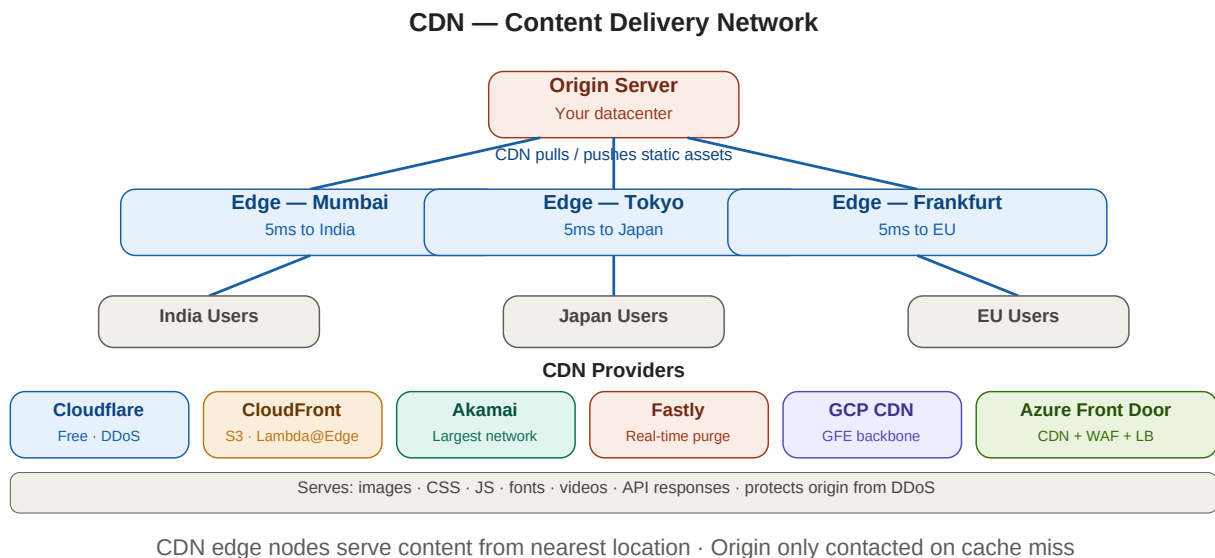
Technology	Type	Throughput	Persistence	Best for	Key Features
Redis	In-memory KV store	> 1M ops/sec	RDB + AOF snapshots	Sessions, leaderboards, pub/sub	Data structures (sets, sorted sets, lists), Lua scripts, clustering
Memcached	In-memory KV store	Very high	None	Simple KV cache	Multi-threaded, simpler than Redis, no persistence
AWS ElastiCache	Managed Redis/Memcached	Same as above	Redis: yes	Managed, zero ops	Auto-scaling, Multi-AZ, read replicas
Hazelcast	Distributed cache (Java)	High	Yes	Java microservices	Distributed maps, near-cache, compute
Varnish	HTTP reverse proxy cache	Very high	No	HTTP response caching	VCL config, CDN-like edge caching
Caffeine (Java)	In-process JVM cache	Highest (no network)	No	Hot data, local JVM	Near-optimal eviction, async loading

Cache Eviction Policies

Policy	Evicts	Best for
LRU — Least Recently Used	Oldest accessed item	General purpose, most common
LFU — Least Frequently Used	Least-accessed item	When access frequency matters more than recency
TTL — Time To Live	Item after fixed time expires	Data that becomes stale (prices, sessions)
FIFO — First In First Out	Oldest inserted item	Simple queues, sliding window data
Random Replacement	Random item	When miss cost is uniform

6. Step 6: Content Delivery Network (CDN)

Even with fast servers, a user in India hitting a datacenter in Virginia adds **150–250ms of network latency**. A CDN places copies of static assets (images, JS, CSS, videos) at **edge nodes worldwide**. Users hit the nearest edge, cutting latency to <5ms.



CDN Providers — Deep Comparison

Provider	Edge Nodes	DDoS Protection	Dynamic Caching	Best for
Cloudflare	310+ cities worldwide	Yes — included free	Yes (Workers)	Small-large apps, DDoS protection, free tier
AWS CloudFront	600+ PoPs	AWS Shield integration	Lambda@Edge	AWS ecosystem, S3 + API Gateway integration
Akamai	4000+ locations	Yes — enterprise	Yes (EdgeWorkers)	Enterprise, largest network, streaming
Fastly	90+ PoPs	Yes	Yes (Compute@Edge)	Real-time cache purge, dev-friendly config
GCP Cloud CDN	Google's backbone	Cloud Armor	Yes	GCP apps, YouTube-scale video
Azure Front Door	192 PoPs + WAF	Yes	Yes (Rules Engine)	Azure apps, global load balancing + CDN

CDN caches: images, CSS, JS, fonts, video, binary files, and increasingly API responses (dynamic caching).

Cache invalidation: push (you tell CDN to purge) vs pull (CDN fetches from origin on miss, caches TTL).

CDN also protects origin from DDoS — attacker hits edge nodes, not your server.

7. Step 7: Multi-Region & Fault Tolerance

A single datacenter means all global users route to one location and **one failure takes everything down**. Multiple regional datacenters solve both: users in India hit the India DC, users in Japan hit Japan. If one DC fails, DNS routes traffic to the next nearest.

- **GeoDNS routing** — DNS resolves to different IPs based on user's location
- **Active-Active** — all DCs serve traffic; any DC can handle any request
- **Active-Passive** — primary handles traffic; standby takes over on failure
- **Data synchronisation** — writes replicate across DCs (adds latency; consistency tradeoff)
- **Regional caches** — each DC has its own Redis cluster to avoid cross-region cache misses

AWS: deploy to multiple Availability Zones (same region) and multiple Regions for global HA.

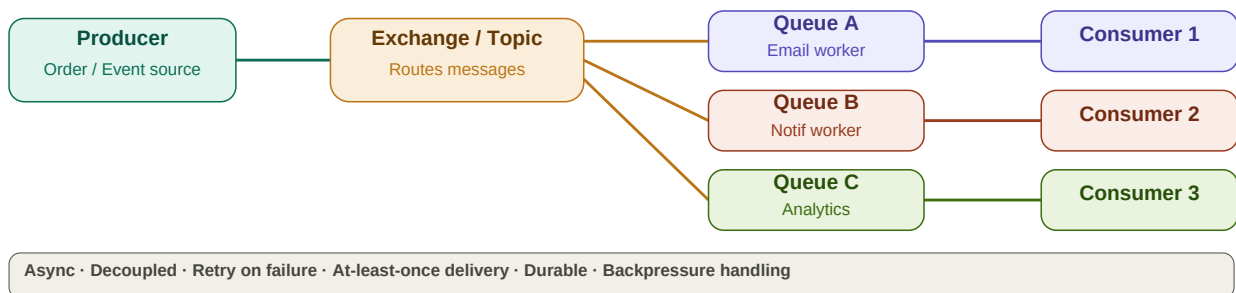
RTO (Recovery Time Objective) and RPO (Recovery Point Objective) drive multi-region decisions.

Cost: multi-region doubles infrastructure — only add when SLA demands it (99.99% uptime).

8. Step 8: Message Queues — Async Processing

At high scale, **synchronous processing creates bottlenecks**. A payment API that also sends email, pushes notifications, updates analytics, and triggers inventory — all in one request — is slow and fragile. Message queues let the API return immediately and **delegate work asynchronously**.

Message Queue — Async Processing Architecture



Message Queue Technologies

Apache Kafka	RabbitMQ	AWS SQS	Celery + Redis
✓ High throughput (M/sec) ✓ Durable log — replay ✓ Partitioned parallelism ✗ Complex ops, high cost Best: streaming, events Used: Netflix, Uber, LinkedIn	✓ Flexible routing ✓ AMQP · easy setup ✓ Dead letter queues ✗ Lower throughput Best: task queues, notifs Used: many web apps	✓ Fully managed ✓ No infra ops ✓ FIFO + standard ✗ AWS lock-in ✗ No event replay Best: AWS-native apps	✓ Python native ✓ Simple setup ✓ Good for tasks ✗ Python only ✗ Not durable Best: Python bg jobs

Producer publishes events · Exchange routes to queues · Consumers process independently and asynchronously

Message Queue Technologies — Deep Comparison

Technology	Model	Throughput	Ordering	Reply	Best for
Apache Kafka	Distributed log (topics/partitions)	Millions/sec	Per-partition	Yes — indefinite	Event streaming, audit logs, microservices events, CDC

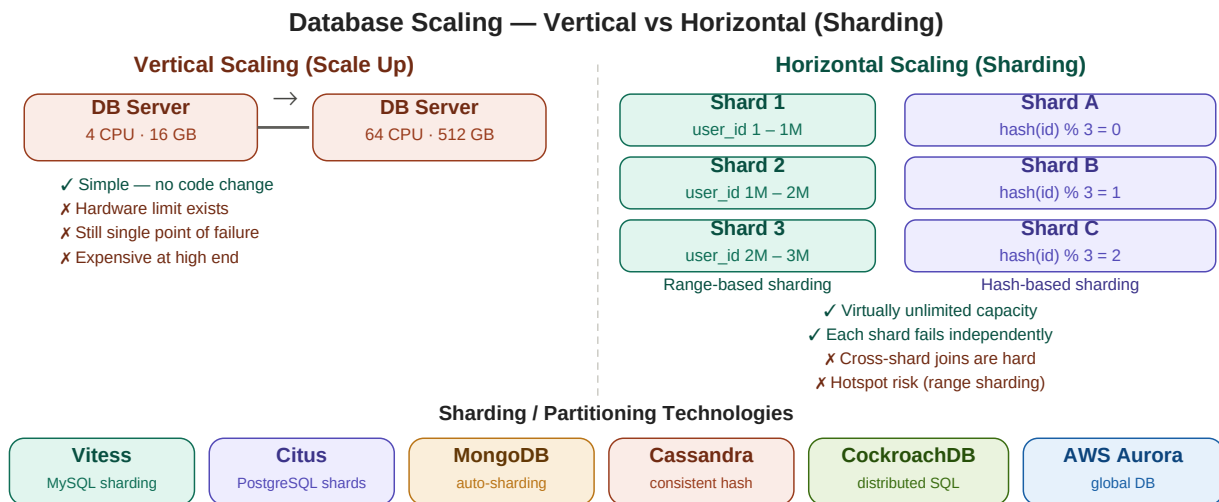
Technology	Model	Throughput	Ordering	Replay	Best for
RabbitMQ	AMQP broker (exchanges/queues)	100K–500 K/sec	Queue-level	No (delete on ack)	Task queues, background jobs, routing, notifications
AWS SQS	Managed queue (standard/FIFO)	~3000/sec FIFO, unlimited standard	FIFO mode only	No	AWS-native task queues, decoupling lambdas
AWS SNS	Pub/Sub fanout	High	No	No	Broadcast events to multiple SQS/Lambda/HTTP subscribers
Google Pub/Sub	Managed pub/sub	Very high	No	Yes (7 days)	GCP apps, streaming to BigQuery, dataflow
Redis Streams	Append-only log	Very high	Yes (per stream)	Yes	Lightweight Kafka alternative, same-server producers
Celery + Redis/RabbitMQ	Task queue (Python)	Medium	Queue-level	No	Python background jobs, scheduled tasks, workers
ActiveMQ	AMQP/STOMP/MQTT broker	Medium	Yes	No	Enterprise Java, IoT MQTT, legacy JMS apps

Key Message Queue Concepts

Concept	Definition
Producer	Service that publishes messages (e.g. Order Service publishes order_created)
Consumer	Service that reads and processes messages (e.g. Email Service)
Exchange (RabbitMQ)	Routes messages to queues based on routing keys and binding rules
Topic (Kafka)	Named log stream; consumers subscribe to topics; messages stored by partition
Consumer Group (Kafka)	Multiple consumers sharing a topic; each partition assigned to one consumer
At-least-once delivery	Message guaranteed to arrive but may duplicate — consumer must be idempotent
Exactly-once delivery	Message delivered exactly once — hard to achieve; Kafka has transactions for this
Dead Letter Queue (DLQ)	Queue where failed messages go after max retry — inspect and replay manually
Backpressure	Producer slows down when consumers can't keep up — prevents queue overflow

9. Step 9: Database Scaling — Sharding

Even with master-slave replication, the master becomes a **write bottleneck** at extreme scale. Sharding (horizontal partitioning) splits the dataset across multiple independent database nodes — each node owns a **subset of the data**.



Vertical scaling hits hardware limits · Range sharding splits by key range · Hash sharding distributes evenly

Sharding Strategies

Strategy	How	Pros	Cons	Hotspot Risk
Range-based	Shard by key range (user_id 1–1M → Shard 1)	Simple, range queries fast	Uneven load if data skews	High (if one range is hot)
Hash-based	Shard = hash(key) % N	Even distribution	No range queries	Low
Directory-based	Lookup table maps key → shard	Flexible resharding	Lookup overhead, SPOF	Low
Geographic	Shard by user region	Low-latency for region	Cross-region queries hard	Medium

Sharding Technologies

Technology	Database	Sharding Model	Best for
Vitess	MySQL	Range/hash-based	YouTube uses it; MySQL at massive scale
Citus	PostgreSQL	Hash/range distribution	Postgres sharding with full SQL
MongoDB	MongoDB	Ranged + hashed sharding	Built-in, auto-balances shards
Cassandra	Cassandra	Consistent hash ring	Leaderless, write-heavy, always-on
CockroachDB	Distributed SQL	Automatic range splitting	Distributed ACID transactions
AWS Aurora Sharding	MySQL/PostgreSQLs	Manual via application	Managed, multi-tenant SaaS

Technology	Database	Sharding Model	Best for
TiDB	MySQL-compatible	Raft + automatic sharding	HTAP (hybrid transactional + analytical)

Cross-shard joins: avoid by denormalizing data or using CQRS — read models aggregate across shards.

Resharding: adding a shard requires rebalancing — consistent hashing minimises data movement.

Hotspot: hash-based avoids hotspots; range-based is vulnerable if one range is disproportionately active.

10. Quick Revision & Interview Questions

Quick Revision

- Step 1–2: Separate app server and DB server — independent scaling, no direct DB exposure.
- Step 3: Load balancer distributes traffic, hides server IPs, enables horizontal scaling of app tier.
- Step 4: Master handles writes, slaves handle reads — reduces read bottleneck, enables failover.
- Step 5: Cache (Redis/Memcached) stores hot data in memory — check cache first, DB on miss only.
- TTL ensures cache entries expire — prevents serving stale data indefinitely.
- Step 6: CDN serves static assets from nearest edge node — reduces latency from 200ms to <5ms globally.
- Step 7: Multi-region deploys — GeoDNS routes to nearest DC; if one region fails, others take over.
- Step 8: Message queues (Kafka/RabbitMQ) decouple producers and consumers — async, retry, durable.
- Use Kafka for event streaming at scale. Use RabbitMQ for flexible task queues. Use SQS for AWS apps.
- Step 9: Vertical scaling hits hardware limits. Horizontal sharding gives unlimited capacity.
- Range sharding risks hotspots. Hash sharding distributes evenly but breaks range queries.
- Cache-aside is the most common pattern — app manages cache population explicitly.
- Replication lag: always read your own writes from master, not slave, immediately after a write.
- Dead letter queue: failed messages after max retries — inspect and replay to debug issues.

Interview Questions

- 1. Walk me through scaling a system from 0 to 1 million users step by step.
- 2. Why separate the application server and database server?
- 3. What is the difference between L4 and L7 load balancing?
- 4. How does master-slave database replication work? What are the failure scenarios?
- 5. What is replication lag and how do you handle it?
- 6. What is the difference between Redis and Memcached?
- 7. Explain cache-aside vs write-through vs write-behind strategies.
- 8. What happens on a cache miss? How do you prevent a thundering herd?
- 9. What does a CDN cache? When does it go to the origin server?
- 10. What is the difference between Kafka and RabbitMQ? When do you use each?
- 11. What is at-least-once delivery? How do you handle duplicate messages?
- 12. Explain range-based vs hash-based sharding. Which prevents hotspots?
- 13. How do you handle cross-shard transactions or joins?
- 14. What is a dead letter queue and when do you use it?

■ 15. How would you design a notification system that handles 10M events/day?

Key Terms

Load Balancer	Distributes incoming traffic across multiple servers using algorithms (RR, Least Conn)
L4 Load Balancer	Routes at TCP/IP level — fast, no content inspection, used for raw throughput
L7 Load Balancer	Routes at HTTP level — path/host-based routing, SSL termination, smarter
Master DB	Handles all write operations (INSERT, UPDATE, DELETE)
Slave / Replica DB	Read-only copy of master — serves SELECT queries to reduce master load
Cache	In-memory store of hot data — eliminates expensive DB queries
TTL	Time To Live — cache entry expires after N seconds, forcing fresh DB fetch
Cache-Aside	App checks cache → miss → fetch from DB → store in cache
Cache Miss	Requested key not in cache — must fetch from DB
CDN	Network of edge servers that serve static assets from nearest geographic location
GeoDNS	DNS that resolves to different IPs based on user's geographic location
Message Queue	Async buffer between producers and consumers — decouples systems, enables retry
Producer	Service that publishes messages to a queue or topic
Consumer	Service that reads and processes messages from a queue
DLQ	Dead Letter Queue — receives messages that failed max retry attempts
Sharding	Horizontal partitioning — split data across multiple independent DB nodes
Range Sharding	Partition by key range — fast range queries but hotspot risk
Hash Sharding	Partition by hash(key) — even distribution, no range queries
Thundering Herd	Many requests hitting DB simultaneously on cache expiry — mitigate with cache lock
Replication Lag	Delay between master write and slave sync — read-your-write issues